

Low-Latency Cross-Venue Arbitrage: Binance $\rightarrow$ Polymarket

Strategy, Infrastructure, Deployment, and Diagnosis of a Live Failure

Cullen Trace Campana

Abstract. This report documents the design, deployment and diagnosis of a real-money low-latency trading system that traded the repricing lag between Binance spot and Polymarket’s short-dated crypto prediction markets. The system spans two AWS regions. A Tokyo node in the same region as Binance detects order-book moves with per-asset gradient-boosted models and transmits a 20-byte, loss-tolerant UDP signal to a Dublin node in the same region as Polymarket, which validates against the live CLOB and executes fill-and-kill orders. End-to-end latency, dominated by the transatlantic hop, was measured at 140–150 ms against a repricing process with a median half-life of roughly 786 ms.

A replay backtest over 8,576 trades won 61.4% with conditional structure consistent with a microstructure edge. Deployed with real capital at \$5 per trade on approximately \$100 per month of infrastructure, the strategy came out flat. The live fill rate was roughly 10% against approximately 100% in paper, and the fills that did occur were close to coin flips. The displayed depth the strategy was built to take was not available to a taker arriving at 150 ms. This report separates what the data can establish about that outcome from what it cannot, reviews three candidate mechanisms, and sets out what a corrected validation process would have required before capital was committed.

Contents

1	Executive Summary	3
1.1	Backtest and Live Results	3
1.2	What the Data Does and Does Not Establish	3
1.3	Reading the Backtest	3
2	The Trade Idea	4
2.1	Polymarket Short-Dated Crypto Markets	4
2.2	Where the Edge Comes From	4
2.3	The Repricing Process, Measured	4
2.4	Why the Edge Is Structurally Thin	5
3	Signal Detection (Tokyo)	6
3.1	Feed and Rolling State	6
3.2	From Hand-Tuned Gates to a Model	6
3.3	Sizing Signal	7
3.4	Per-Asset Calibration	7
3.5	The Fill Assumption Is Baked Into the Model	7
4	Execution Logic (Dublin)	8
4.1	Entry Gates	8
4.2	Order Execution	8
4.3	Exit Paths	9
4.4	Sizing and Risk Limits	9
5	Infrastructure	11
5.1	Two-Node Topology	11
5.2	The Wire Protocol	11
5.3	Node-Internal Design	12
5.4	Mock-to-Live Parity	13
5.5	Latency Budget	13
6	Deployment	14
6.1	Instances and Placement	14
6.2	Measured Latencies	14
6.3	Operations	14
7	Backtest Performance	15
7.1	Headline Results	15
7.2	Fees Dominate	15
7.3	Conditional Structure	16
7.4	The Tick-Rate Effect: Reversal, Not Throughput	17
8	Live Results	19
8.1	What Happened	19
8.2	Attempted Remediation	19
8.3	The One Fact the Data Establishes	19

9	Candidate Mechanisms	20
9.1	Quote Fading by Faster Participants	20
9.2	The Off-Chain to On-Chain Settlement Gap	20
9.3	Spoofing	20
9.4	Why the Paper Harness Could Not Distinguish Them	21
9.5	Adverse Selection Among the Fills That Occurred	21
10	Lessons	22
10.1	Realisable Liquidity Is a Measurement, Not an Assumption	22
10.2	A Fill Model Must Be Able to Decline	22
10.3	A Learned Detector Inherits Whatever the Labels Assume	22
10.4	Geography Is Computed First	22
10.5	Calibrate Against the System That Will Run	23
10.6	Where This Approach Does Work	23
11	Conclusion	24
A	System Reference	25
A.1	Component Map	25
A.2	Results Summary	26
A.3	Key Parameters	27
A.4	Data Provenance	27

1 Executive Summary

Polymarket’s short-dated crypto “up-or-down” markets reprice with a measurable lag relative to the underlying spot price on Binance. This report documents a system built to trade that lag, the evidence that the lag is real, and the reasons the strategy could not capture it with live capital.

The system was engineered to production depth: two compute nodes in different AWS regions, a 20-byte UDP signalling protocol, shared-nothing per-asset processing pipelines, a mock-to-live execution harness, and per-asset calibration against replayed market data. It was deployed to AWS and traded real capital at \$5 per trade before being retired. The project spanned about two months end to end.

1.1 Backtest and Live Results

In replay, across 8,576 trades drawn from roughly 52 hours of recorded order flow, the strategy won 61.4% of the time with a positively skewed per-trade P&L distribution. The conditional structure was coherent: performance was strongest in quiet books, at prices away from 0.50, and at short holding periods, and weakened in each of the opposite conditions. Fees consumed 55% of gross P&L.

Live, roughly one marketable order in ten filled, against nearly all of them in paper. The signal, the gates and the latency were the same in both. The fills that did occur were close to coin flips, and the deployment finished flat. The whole effort ran to about two months, after which I retired it and moved on to a cross-venue relative-value system instead.

1.2 What the Data Does and Does Not Establish

Three things are on firm ground. Polymarket reprices toward the signal after a Binance move, with a median half-life near 786 ms (Section 2.3). Orders arrived inside the latency budget, at 140–150 ms (Section 5.5). And the depth shown on the Polymarket book was, for the most part, not there to take at that latency (Section 8).

Why it was not there is the open question. Distinguishing a maker who faded faster than I arrived from one whose resting size was never firm needs quote-lifecycle or on-chain settlement data for my own orders, and I did not keep it. Section 9 lays out the three candidates and is honest about which of them the surviving data can and cannot rule out.

1.3 Reading the Backtest

All backtest results are paper results computed under a displayed-liquidity fill assumption, reconstructed from recorded feed sessions (the `regime_trade_log` and `edge_signals` datasets). That assumption is the one that failed live, so every figure in Section 7 describes the strategy’s behaviour conditional on displayed depth being available, not its expected live performance.

2 The Trade Idea

2.1 Polymarket Short-Dated Crypto Markets

Polymarket lists a continuous series of short-horizon binary markets on major cryptocurrencies. For each asset in {BTC, ETH, SOL, XRP} and each timeframe in {5 min, 15 min}, a market opens asking whether the asset will close the window above or below its opening reference. Each market resolves to a pair of outcome tokens (“Yes” for up, “No” for down) that trade on a central limit order book priced in $(0, 1)$ and settle at \$1 for the correct outcome and \$0 for the incorrect one. A new window rotates in every 5 or 15 minutes. The system discovers active windows by constructing the market slugs (`btc-updown-5m-{ts}`) and querying Polymarket’s Gamma API.

Only the 5-minute markets were traded. They were selected first because the higher window-rotation frequency produces more repricing events per hour, and because they were more profitable in paper. The 15-minute markets were implemented and available but never traded live, so no live evidence exists for them and no claim is made about them here.

2.2 Where the Edge Comes From

Let P_t be the Binance mid price of the underlying, R the window’s reference level, and τ the time remaining. The fair value of the up token is approximately

$$V_t^{\text{up}} \approx \Pr[P_{t+\tau} > R \mid P_t],$$

which, over these horizons, is a smooth increasing function of the distance $P_t - R$. The quantity V_t^{up} responds to P_t instantaneously. The Polymarket order book responds only through the actions of participants watching Binance and re-quoting, which takes time.

2.3 The Repricing Process, Measured

The lag is visible in the recorded data. For every detected Binance move, the `edge_signals` dataset records the Polymarket mid at fixed horizons afterward and computes how far the Polymarket price moved in the signalled direction. Figure 1 shows the pooled and per-asset averages.

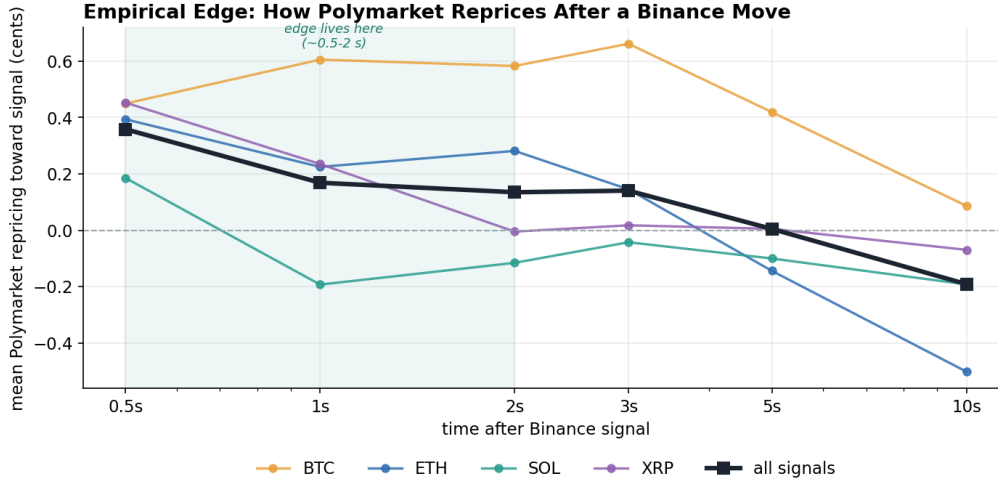


Figure 1: Empirical Polymarket repricing (in cents, in the signalled direction) at fixed horizons after a Binance move, per asset and pooled. Repricing begins within the first few hundred milliseconds, is largely complete by roughly two seconds, and mean-reverts past its starting point by ten seconds.

Repricing is a process, not an event, and the timing is worth pinning down because the latency argument later depends on it. The first quote updates land within roughly 150–500 ms. The median half-life is around 786 ms. Most of the move is done by two seconds, and by ten seconds the price has decayed back past where it started.

This is why 150 ms is both fast and slow. An order arriving then beats the full two-second repricing, so there is still displacement left to capture, but it arrives alongside the first re-quoters rather than ahead of them, and they are the participants setting the new price. Fast enough to beat the crowd, not the front of it. Section 5.5 comes back to this.

2.4 Why the Edge Is Structurally Thin

The edge is bounded on both sides. The repricing process caps how long the opportunity exists at roughly two seconds. The physical distance between the price source (Binance, reached from Asia) and the execution venue (Polymarket, reached from Europe) imposes a one-way transit of approximately 120 ms on every signal. Transit therefore consumes around 6% of the two-second window but all of the first, most competitive fraction of it, which is where the participants who set the new price are acting. This tension motivated the two-node architecture of Section 5.

3 Signal Detection (Tokyo)

3.1 Feed and Rolling State

Detection runs in Tokyo, in the same AWS region as Binance, so a move is seen as early as placement allows. The node consumes the `depth10@100ms` book snapshot stream and the `@trade` stream for all four assets, about 3,000 updates per second in aggregate, and keeps 60 seconds of rolling per-asset state: mid, order-flow imbalance, VWAP, separated buy and sell volume, book imbalance, and a price ring. The snapshot stream is throttled to 100ms, so between snapshots the price ring is filled in from the `@trade` feed, which is not throttled; the individual prints give a finer-grained view of where the mid went inside each 100ms gap than the snapshots alone would.

3.2 From Hand-Tuned Gates to a Model

The first version of the detector was a conjunction of three hand-tuned gates: a minimum Binance price move over a short lookback, a minimum book-event density, and a minimum of confirming signed dollar flow. A signal fired when all three passed. The reasoning was standard enough. A real move should show up as a price change, arrive as a burst of activity, and be backed by executed volume rather than by quotes alone.

This worked less well than the reasoning suggested, and the reason is why the system ended up where it did. The three conditions are not independent. Dollar flow, tick density and move size rise and fall together, and they do so differently in a quiet book than in a fast one, so a fixed threshold on each was either too loose in active periods or too tight in quiet ones, and tuning one gate moved the effective operating point of the others. Treating three correlated features as independent switches threw away most of the information in how they co-moved.

The gates were therefore replaced by per-asset, per-timeframe gradient-boosted models (LightGBM, with random forests as a cross-check), which is what ran live. The model takes the full microstructure feature vector at each candidate tick, the same quantities the gates used plus volatility ratios, book depth, tick and trade rates, and the Polymarket spread and distance from 0.50, and outputs the probability that the tick converts into a profitable trade. It learns the joint structure the gates could not: that a given move size means one thing at 20 ticks/s and another at 300, and that flow and density matter conditionally on each other rather than additively.

The old gates survive only as a cheap pre-filter. A coarse move-size threshold runs first and discards the large majority of ticks that are obviously nothing, so the model is scored only on the small fraction that might be a signal and never sits in the hot path for a quiet tick. Everything downstream of that pre-filter is the model.

A caveat on detection latency

Because the book stream is throttled to 100ms, a move can have happened up to 100ms before the snapshot that first shows it in the book. The trade-feed interpolation recovers some of this, since prints arrive between snapshots, but not all of it. The latency budget in Section 5.5 is measured from snapshot receipt onward, so it does not include this term, and the true worst case from market event to order arrival is up to 100ms longer than the figures quoted there. Subscribing to the raw un-throttled diff stream would have closed most of the gap. Not doing so was a mistake, and an avoidable one.

3.3 Sizing Signal

The model’s output probability doubles as a sizing input and is passed to Dublin in the signal packet. A separate minimum-confidence entry gate of 0.10 exists on the Dublin side; in practice the model rarely emits anything that low on a tick that has already cleared the pre-filter, so that gate almost never binds. It is a leftover from the gate-based scheme and is closer to dead code than to a real control.

3.4 Per-Asset Calibration

Each asset gets its own model and its own set of execution parameters, because tick size, volatility and typical flow differ enough across BTC, ETH, SOL and XRP that a shared configuration fits none of them well. Models are trained offline on recorded sessions; the execution and exit parameters around them are searched with Optuna’s TPE sampler (150 trials per asset) against a replay of the same recorded data. Table 1 shows representative execution parameters.

Table 1: Representative per-asset execution parameters (excerpt).

Asset	Pre-filter thr.	Flow min.	Cooldown	Exit timeout
	(bps)	(USD)	(s)	(s)
BTC	4.75	15,000	5.0	14
ETH	3.75	800	4.5	27
SOL	4.75	11,800	10.0	3
XRP	5.00	13,800	1.0	27

The exit timeouts in the last column turn out to be misconfigured against the exit logic that consumes them, which Section 4 works through.

3.5 The Fill Assumption Is Baked Into the Model

There is a catch in using a learned detector here, and it turned out to matter more than any of the architecture. The model predicts *profitable trades*, and profitability in the training labels was computed under the paper fill assumption, against the displayed book. Once that assumption failed live (Section 8), every label the model had learned from was defined against an execution surface that did not exist, so the model had effectively been trained to spot ticks that would have paid off in a market that was not the real one. Retraining on the same labels could not fix that, because the problem sits upstream of the model rather than inside it.

4 Execution Logic (Dublin)

The Dublin node turns a directional signal into a filled position. Its design priority is the inverse of Tokyo’s: Tokyo optimises for the earliest possible detection, Dublin for declining to trade unless conditions are favourable, since on a thin edge a bad fill erases many good ones.

4.1 Entry Gates

Every incoming signal passes through ten gates, checked in order and short-circuiting on the first failure:

1. **Signal age.** Reject signals older than 500 ms.
2. **Clock skew.** Reject signals timestamped more than 50 ms in the future, indicating clock desync or corruption.
3. **Confidence.** Reject signals below 0.10. As Section 3 notes, the model almost never emits a score this low on a tick that has cleared the pre-filter, so this gate rarely bites.
4. **No-trade zone.** Refuse to open within 30 s of window expiry, where resolution risk dominates the repricing edge.
5. **Book present.** There must be a live Polymarket book for the target token.
6. **Spread.** Reject if the Polymarket bid/ask spread exceeds approximately 4–5¢, where crossing costs consume the edge.
7. **Book staleness.** Reject if the newest book snapshot is older than 5 s.
8. **Price bounds.** Only trade when the outcome token’s ask is in $[0.25, 0.80]$.
9. **Dynamic threshold.** A signal-strength-scaled price check.
10. **Position and fill.** The slot must be flat and within risk limits, and the fill attempt itself must succeed.

The price-bounds gate deserves comment. Section 7.3 shows that outcomes near 0.50 are close to coin flips, but $[0.25, 0.80]$ contains 0.50 and therefore does not exclude that region. The gate excludes the tails, where the token price is insensitive to the underlying, which is a different objective. A gate on distance from 0.50 would have matched the empirical finding; the asymmetry of the bounds was inherited from early manual tuning and was never revisited.

4.2 Order Execution

Orders are executed as fill-and-kill (FAK) marketable orders, Polymarket’s immediate-or-cancel, which fill at the quoted price or better and cancel any unfilled remainder. A single FAK priced at the second book level sweeps the top level first, since it is a better price, then fills into the second, capturing both levels with one order and no stepping logic. Buys are priced to take displayed liquidity. Sells, where exiting matters more than price, are priced more aggressively and post a resting good-till-cancelled remainder if the sweep leaves size unfilled.

Polymarket’s taker fee for these markets is modelled per contract as

$$f(p) = p \cdot 0.25 \cdot (p(1-p))^2,$$

which peaks near the middle of the price range and vanishes at the tails.

4.3 Exit Paths

A position is closed by whichever of five conditions fires first:

1. **Stop-loss.** The outcome token's bid has collapsed past a threshold (20–30% of entry).
2. **Reversal.** The Binance underlying has reversed through a per-asset threshold, invalidating the thesis at the source.
3. **Convergence.** After a minimum hold of approximately 3.5s, the Polymarket mid has moved through entry in our favour and Binance velocity has exhausted. This is the intended exit and accounts for 78% of closes.
4. **Timeout.** A per-asset maximum hold (3–27s) has elapsed.
5. **Emergency.** A 15s dead-man timer that force-closes regardless of state.

Two configuration faults

Reviewing these parameters against Table 1 surfaces two faults that were live throughout.

First, the 15s dead-man timer preempts any per-asset timeout longer than 15s. The calibrated 27s timeouts for ETH and XRP could therefore never fire, and positions in those assets that would have been held to the calibrated horizon were force-closed at 15s instead. Some fraction of the 16% of closes attributed to the emergency path are calibrated timeouts in disguise.

Second, SOL's calibrated timeout of 3s is shorter than the 3.5s minimum hold required before a convergence exit can trigger. SOL positions could therefore never exit via the intended convergence path. Any SOL profitability in Section 7 was achieved despite this, not through the mechanism the strategy was designed around.

Both faults follow from calibrating detector and exit parameters against a replay that did not enforce the live state machine's constraints. The calibration and the execution logic were validated separately and never against each other.

4.4 Sizing and Risk Limits

Sizing is small and bounded, appropriate to a strategy whose capacity is unknown and whose fills are uncertain. Paper trading used \$100 per order; live trading used \$5 per order, with a hard ceiling of 200 contracts per trade. Exposure is capped at \$500 per asset and \$2,000 in aggregate, with no more than 10 open orders at once. A dual-window rate limiter (a 10-second burst bucket and a 10-minute sustained bucket, mirroring Polymarket's published limits) gates every API call and always reserves tokens for closes, so the system cannot rate-limit itself out of exiting a position.

These are caps rather than realised sizes. Because entry orders are fill-and-kill sweeps of the top book levels, the filled quantity is whatever depth happens to be resting, and the \$100 paper parameter was rarely the binding constraint. Realised notional per trade was therefore a small multiple of a dollar even in paper, which Section 7 quantifies. This matters for interpreting every P&L figure in this report: the cent totals are sums over trades of a few contracts each, not over \$100 positions.

The size reduction from paper to live was deliberate, to bound the cost of discovering an unknown. It also means the live sample carries less statistical weight per trade and that live and paper results are not directly comparable in absolute P&L, only in fill rate and win rate.

5 Infrastructure

5.1 Two-Node Topology

Because Binance and Polymarket are reached from different continents, detection and execution are split across two nodes (Figure 2). The Tokyo node does all Binance-facing work and no execution; the Dublin node does all Polymarket-facing work and no detection. They communicate over a single connectionless link, so each node sits close to the venue it talks to and the intercontinental hop is crossed once, by the smallest possible message, as early as possible.

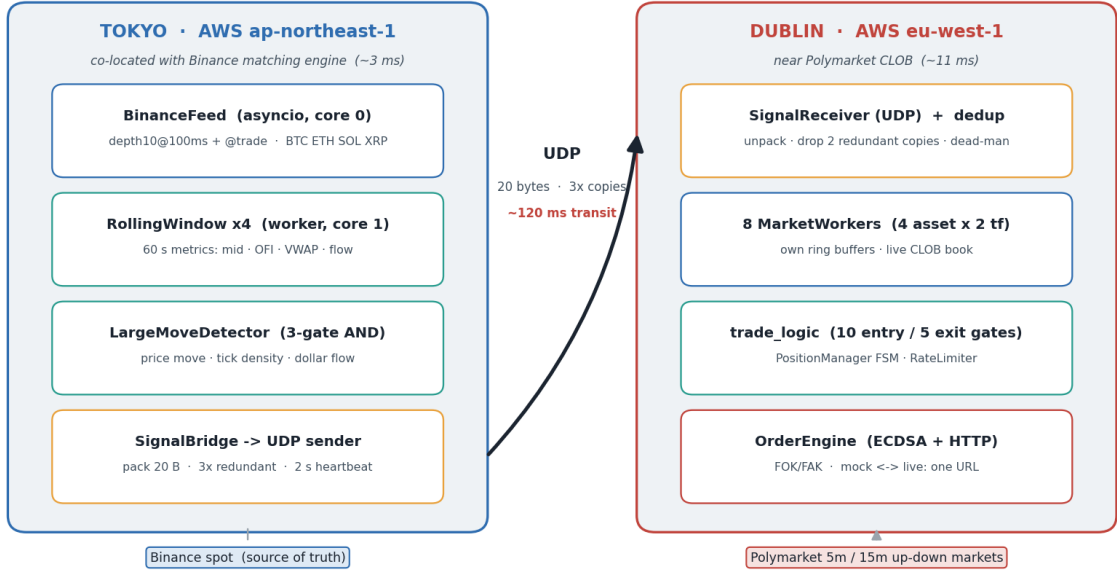


Figure 2: Two-node architecture. Tokyo detects Binance moves and transmits a 20-byte UDP signal; Dublin receives, validates against the live Polymarket book, and executes.

5.2 The Wire Protocol

The link is a hand-rolled binary protocol over UDP. TCP was rejected deliberately: a lost packet on a 120 ms link triggers a retransmit that arrives long after the edge is gone, so reliability through retransmission has no value here. Each signal is packed into 20 bytes and sent as three redundant copies; the receiver processes the first to arrive and discards duplicates by sequence number. This trades bytes for resilience to single-packet loss without waiting for a round trip. The frame is given in Table 2.

Table 2: Signal wire format (20-byte payload, big-endian). One frame carries either an OPEN or a CLOSE; bytes 13–14 are repurposed as a reference id on CLOSE.

Offset	Field	Bytes	Description
0	magic	1	0xA7 marks a valid packet
1	seq	2	sequence number (wraps at 65535)
3	copy_id	1	redundant copy index (0, 1, 2)
4	timestamp	8	epoch microseconds
12	flags	1	bit-packed (below)
13	magnitude	1	OPEN: move in bps / CLOSE: ref id high
14	confidence	1	OPEN: score 0–255 / CLOSE: ref id low
15	price	4	Binance price $\times 10,000$
19	checksum	1	XOR of bytes 0–18

The single `flags` byte packs everything needed to route and interpret the signal: a debug bit, a two-bit close reason, a two-bit asset selector, a direction bit and an action bit. A heartbeat frame (magic 0xA8, sent every 2 s) lets Dublin detect if Tokyo goes dark: if heartbeats stop, Dublin knows no signals are coming and existing positions ride out their timeouts. When Tokyo sets the debug bit, Dublin forces paper mode and captures extended diagnostics, so one flag on the sending side switches the whole system between live and instrumented dry-run.

5.3 Node-Internal Design

The system is written in Python, with latency-critical components in Rust, notably ECDSA order signing. Both nodes follow the same discipline: the thread doing network I/O does no heavy computation, and the thread doing computation does no I/O, with a single non-blocking queue between them.

Tokyo runs an asyncio event loop on one core (Binance WebSockets in, status logging) and a worker thread on another (four rolling-window detectors, exit monitoring, signal packing, UDP send). Handoff is an $O(1)$ `put_nowait` that cannot block the feed. Dublin runs all WebSocket feeds (the Polymarket CLOB, the relayed Binance price stream, and the user fill channel) on its asyncio core, and eight worker threads, one per (asset, timeframe) slot, each owning pre-allocated NumPy ring buffers so that no worker contends with another for its market’s data.

Two caveats on this design. The instances are 2 vCPU, which is two hardware threads of one physical core rather than two independent cores, so the I/O and compute “cores” share execution resources. And because the workers are Python threads, the interpreter lock serialises their Python-level execution; the shared-nothing buffer design eliminates lock contention and cache-line sharing on the data, but not interpreter-level serialisation. Measured node-internal compute was a small number of milliseconds end to end, which was never the binding constraint, so neither was addressed.

Per-slot position state is an explicit finite state machine (`FLAT` $\rightarrow$ `OPENING` $\rightarrow$ `LONG/SHORT` $\rightarrow$ `CLOSING` $\rightarrow$ `FLAT`) locked across the two-phase submit/resolve of every order, which makes concurrent duplicate opens structurally impossible.

5.4 Mock-to-Live Parity

A standalone mock CLOB server replicates Polymarket’s HTTP and WebSocket API: the same order endpoints, the same 200-status-with-error-body convention, the same published rate limits, and order books populated from Polymarket’s real market data feed. The mock subscribes to the live Polymarket book over WebSocket and simulates fills by walking the displayed levels. The Dublin system talks to the mock or to production through the same code path; going live is a change of host URL.

The parity goal was met at the API level. The fill model was not a model of the live venue, however, because it filled against displayed depth. Section 8 covers the consequences.

5.5 Latency Budget

Figure 3 decomposes the measured path from Binance snapshot receipt to order arrival at Polymarket. Round-trip network latency from the Tokyo node to Binance is approximately 3 ms; in-node compute in Tokyo is under a millisecond; the UDP hop to Dublin is approximately 120 ms; Dublin’s gate evaluation and ECDSA signing add a few milliseconds; and the order reaches Polymarket in approximately 11 ms. The total is 140–150 ms.

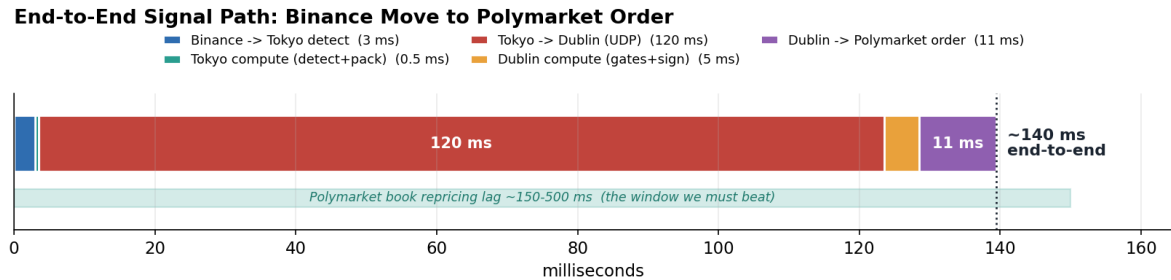


Figure 3: Measured end-to-end latency from a Binance snapshot to a Polymarket order. The approximately 120 ms Tokyo→Dublin transit dominates; all node-internal compute together is a few milliseconds. The 100 ms snapshot granularity discussed in Section 3 is not included.

Against a repricing process with a median half-life of 786 ms, arriving at 150 ms means arriving before most of the repricing. Against the first re-quoters, who begin updating within 150–500 ms, it means arriving at roughly the same time they do. The system was fast enough to beat the crowd and not fast enough to beat the front of it, and the transatlantic segment, which cannot be optimised locally, is what placed it there.

6 Deployment

6.1 Instances and Placement

The system ran on two AWS `c6a.large` instances (2 vCPU each, roughly \$52 per month on demand), one in `ap-northeast-1` (Tokyo) and one in `eu-west-1` (Dublin). Placement means the same AWS region and availability zone as each venue’s endpoints, with inter-node connectivity over AWS’s internal backbone rather than the public internet. This is not colocation in the exchange sense, which would require a cross-connect in the venue’s own facility; it is the closest approximation available without that cost and lead time. The backbone gives lower and, more importantly, lower-jitter latency than a commodity path. Two vCPUs matches the two-thread design of each node. Total infrastructure cost was approximately \$100 per month.

6.2 Measured Latencies

From the deployed nodes, measured characteristics were approximately 3 ms Tokyo to Binance, 120 ms Tokyo to Dublin, and 11 ms Dublin to Polymarket. These are the figures behind Figure 3. They were stable enough that transit variance was never the binding constraint.

6.3 Operations

Each node logs a compact status line at a fixed interval: Tokyo every 5 s (queue depth, signals sent and dropped, per-asset mid, order-flow imbalance, recent move, book imbalance), Dublin every 15 s (fills, cancels, timeouts, exposure, realised P&L, open orders, rate-limit headroom, heartbeat age). Every state change is written to a dual JSONL and parquet trade log, and a post-session analyser computes P&L, win rate, per-asset breakdown and exit-reason distribution at shutdown. The debug flag adds per-event timing histograms and full book-state captures at signal time, at no overhead when disabled.

In hindsight the logging was well suited to diagnosing the system and poorly suited to diagnosing the venue. It recorded what the system did and what the book displayed, but not the maker-level quote lifecycle or the on-chain settlement outcome of each order, which is what Section 9 needs and does not have.

7 Backtest Performance

Results in this section come from replaying the full strategy (model, execution logic, fees and exits) over 13 recorded feed sessions, giving 8,576 trades across roughly 52 hours of 5-minute market order flow. Fills are simulated against the recorded Polymarket book under a \$100 order cap that, as Section 4 notes, rarely bound. As Section 8 explains, the deeper assumption, that displayed depth is there to take, did not hold live, so these numbers describe how the strategy behaves when it is.

7.1 Headline Results

The strategy won 61.4% of its 8,576 trades with a median holding period of 7.6 s. The per-trade net P&L distribution (Figure 4) is centred just above zero and positively skewed, consistent with a small repeatable edge rather than occasional large wins. Exits were dominated by convergence (78%), with the remainder split across the emergency dead-man timer (16%), reversal closes (4%) and stop-losses (2%). Per Section 4, the emergency figure absorbs the ETH and XRP calibrated timeouts, which could not fire.

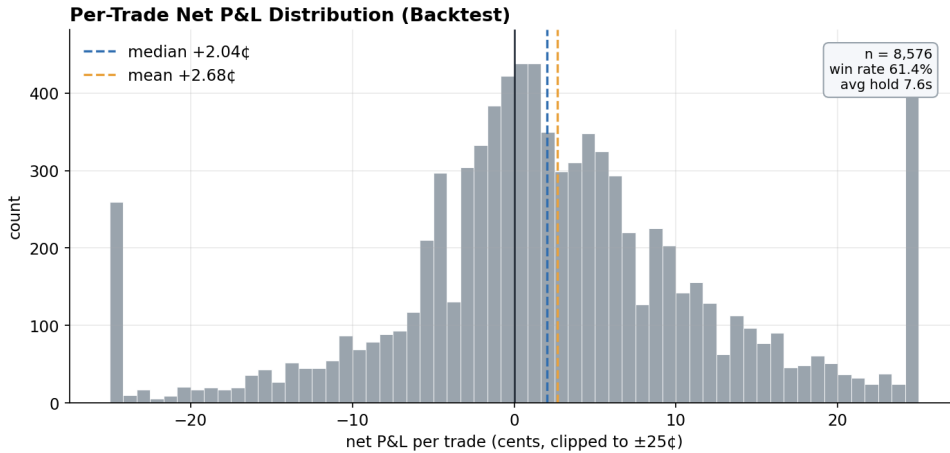


Figure 4: Per-trade net P&L (cents, clipped to $\pm 25\text{¢}$ for display). Positive mean and median, 61.4% win rate, across 8,576 trades.

7.2 Fees Dominate

The largest feature of the equity curve (Figure 5) is the gap between gross and net. The strategy earned +51,350¢ gross and paid 28,403¢ in fees (55% of gross), leaving +22,948¢ net. Across 8,576 trades that is 5.99¢ gross, 3.31¢ in fees and 2.68¢ net per trade.

Realised fill size was not logged per trade, but it can be recovered from the fees. The per-contract taker fee $f(p)$ averages about 0.51¢ over the traded price range (integrating f over $[0.10, 0.90]$, since the strategy did not trade the outer ten cents on either side, and dividing by the width). Set against the logged 3.31¢ of fees per trade, that implies roughly six contracts filled per trade, or a couple of dollars of notional. The \$100 paper order parameter was therefore nowhere near binding: a FAK sweep took whatever thin size rested at the top levels and cancelled the rest.

At six contracts, gross edge is about 1.0¢ per contract and net edge about 0.4¢, which lines up with the sub-cent repricing magnitudes in Figure 1. That the P&L reconciles with the measured repricing is a useful sanity check that it comes from the intended mechanism.

Two things follow. The strategy is fee-limited, not signal-limited: over half of each unit of gross edge goes to fees, so the fee schedule is the first thing that would have to change for a second attempt to make sense. And capacity is small by construction. Even in paper, against a book assumed fully available, a top-of-book sweep cleared only a handful of contracts, so the displayed depth in these markets was thin before any question of whether it was genuine. A strategy that cannot clear size against a cooperative book will not clear it against an uncooperative one.

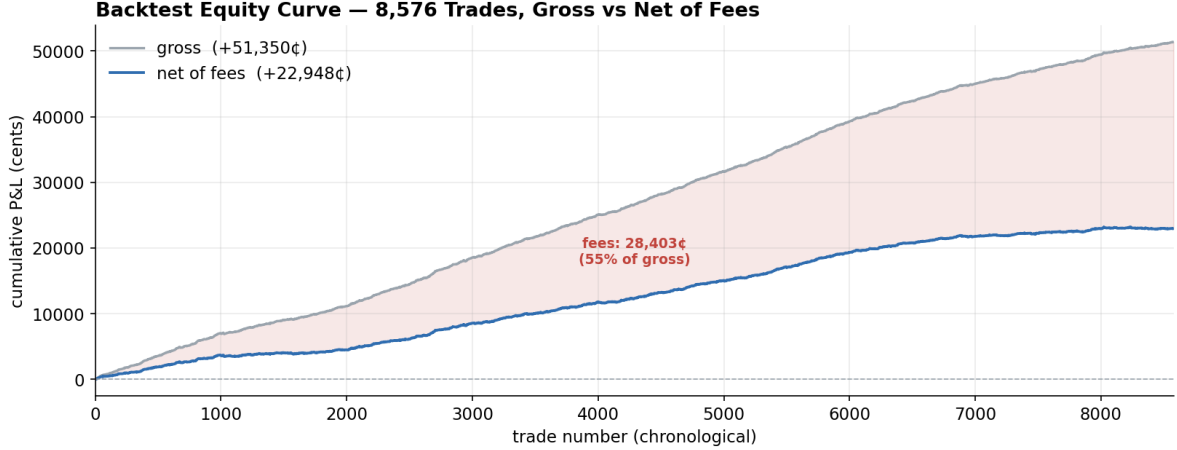


Figure 5: Backtest equity curve over 8,576 chronological trades. Gross P&L (grey) rises steadily; fees (shaded) consume 55% of it, leaving the net curve (blue).

7.3 Conditional Structure

The edge varies systematically with conditions observable at entry (Figure 6), and each relationship has a microstructural reading.

- **Tick rate (panel a).** Win rate falls monotonically from 68% in quiet books (< 30 ticks/s) to a losing 49% in the fastest (> 300 ticks/s). Section 7.4 decomposes this.
- **Distance from 0.50 (panel b).** Near 0.50 the outcome is close to a coin flip (51%); the edge strengthens toward the wings (65–68%), where the token price is more sensitive to the underlying and the fair-value decoupling from a Binance move is larger.
- **Hold time (panel c).** P&L is bimodal. The profitable cohorts exit at 2–5 s (+19,774¢) and 8–12 s (+14,243¢); trades that run into the 12–20 s band are heavily negative (−10,531¢). The interpretation is not that holding beyond the two-second repricing horizon is itself unprofitable, since the 8–12 s cohort is the second most profitable. It is that the 12–20 s band is populated by positions that failed to converge and were force-closed by the dead-man timer, which selects for trades whose thesis was already wrong.
- **Asset (panel d).** All four assets were net profitable in replay, with SOL and XRP posting the highest win rates (approximately 65%) and BTC and ETH the largest absolute P&L.

Conditional Structure of the Edge — 8,576 backtest trades

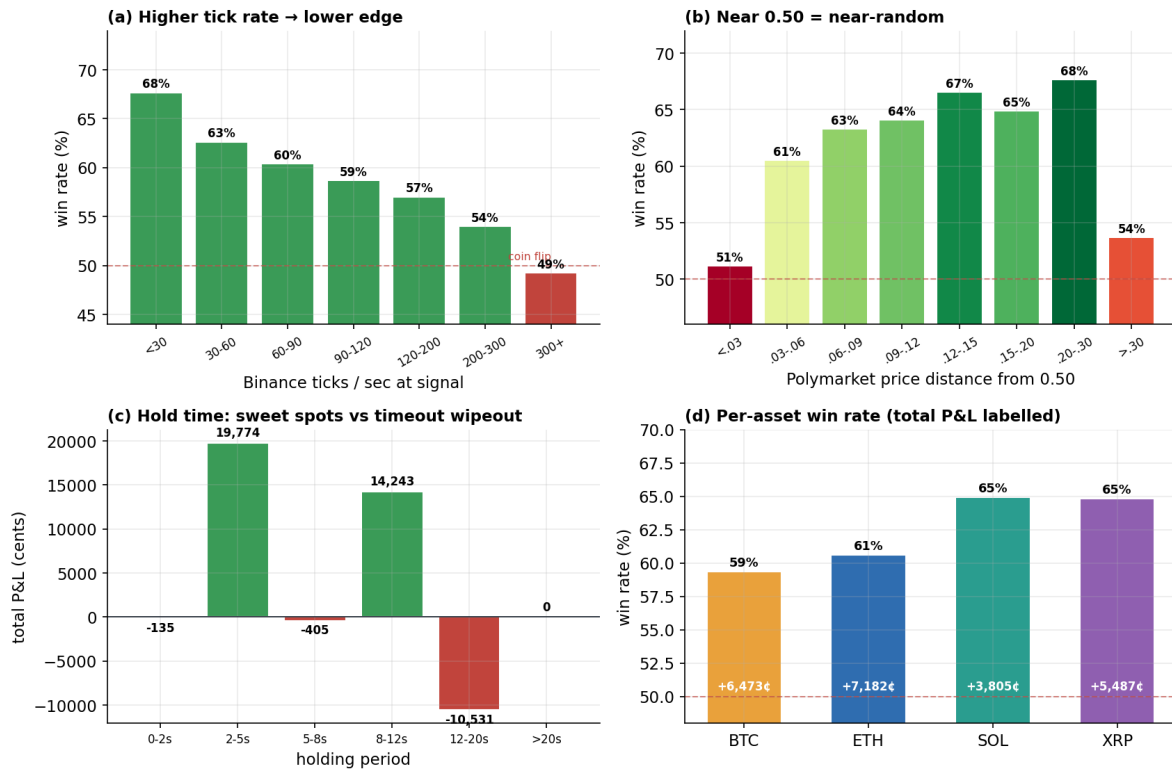


Figure 6: Conditional structure of the edge across 8,576 backtest trades. The edge concentrates in quiet books, away from 0.50, and in trades that converge rather than time out.

7.4 The Tick-Rate Effect: Reversal, Not Throughput

The tick-rate relationship is the strongest of the four, so its cause is worth isolating. One candidate is throughput: that the system cannot keep up when the Binance feed is fast. The data rules this out. The relationship appears in the replay backtest, which processes every recorded tick and does not model the live event queue at all, so it cannot be a throughput artefact. Figure 7 shows the cause has two parts.

Why the edge fades in fast markets — reversal + competition, not throughput

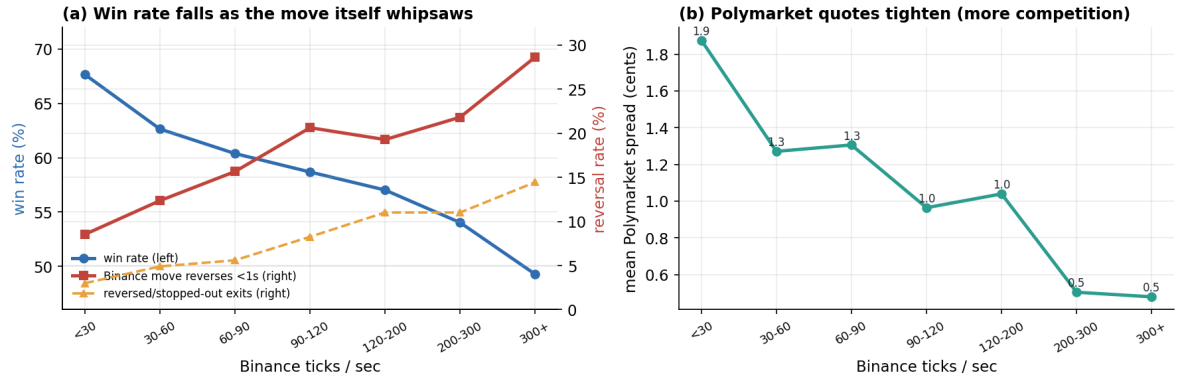


Figure 7: Decomposing the tick-rate effect. (a) Win rate (blue) is close to a mirror image of how often the triggering Binance move reverses within one second (red). Move magnitude is flat at approximately 2.2bps throughout, so reversed and stopped-out exits (amber) climb with tick rate. (b) The Polymarket spread tightens from approximately 1.9¢ to 0.5¢ as activity rises.

Adverse selection at the source is the dominant part. In quiet books (< 30 ticks/s) the triggering move reverses within a second 8.5% of the time; in the fastest (> 300 ticks/s) it reverses 28.6% of the time, a 3.4× increase. Since move magnitude is flat across the range, fast-market signals are more likely to be transient liquidity spikes that snap back, and following momentum into them is adversely selected. Reversed and stopped-out exits rise from approximately 3% to 14% over the same range.

Competition is the secondary part. As Binance activity rises the Polymarket book is quoted more tightly, with mean spread falling from approximately 1.9¢ to 0.5¢, consistent with more market makers and competing latency arbitrageurs being active and a smaller, shorter-lived mispricing remaining.

This matches an operational observation from live trading: results were better during Binance off-peak hours and worse when activity and competition were high. I did not establish the mechanism directly, and the two explanations above are plausible but untested against live data. The backtest supports scaling size down above approximately 120 ticks/s rather than trading flat. No such gate existed in the deployed system; the strategy traded all tick-rate regimes, and this remains an unimplemented recommendation rather than a feature.

8 Live Results

8.1 What Happened

In paper trading against the mock CLOB, the system behaved as Section 7 describes. Signals fired at expected rates, gates rejected stale and thin conditions, and marketable orders filled at approximately 100%, because the mock filled any order the displayed book could satisfy and the displayed book always appeared deep enough.

Live, at \$5 per order over approximately one week, roughly one order in ten filled. Signals, gates and latency were unchanged. The orders that did fill were close to coin flips, and the deployment finished flat. It neither lost meaningful money nor captured the backtested edge.

8.2 Attempted Remediation

I tried two fixes before giving up on it.

The first was to trade deeper into the book, pricing through more levels to reach size that might be firm. Fill rates did improve, but the price concession to reach the deeper levels was larger than the net edge from Section 7. On roughly a cent per contract, crossing one extra level wipes it out, so reaching real liquidity cost more than the liquidity was worth.

The second was to stop depending on latency altogether by predicting the Binance move around 500 ms ahead, which would let orders go in before the fastest re-quoters acted. I built and tested forecasting models for this. The signal at that horizon was too weak to carry a strategy already down to a sub-cent net edge, and I dropped it.

A structural problem compounded the second effort. Every label available for training came from recorded sessions under the displayed-liquidity fill assumption, so the target being predicted was profitability in a simulation whose fill model was known to be wrong. The dataset could not be repaired without a live probe campaign to recalibrate fills, which would have cost more than the expected value of continuing. I retired the strategy at that point.

8.3 The One Fact the Data Establishes

Displayed depth on Polymarket's book was substantially unavailable to a taker arriving at approximately 150 ms. This is directly measured: the same orders, priced against the same displayed levels, filled at approximately 100% against the displayed book and approximately 10% against the real one.

Everything beyond this, in particular *why* the depth was unavailable, is inference, and the next section treats it as such.

9 Candidate Mechanisms

Three mechanisms could produce a 10% fill rate against a book that displays sufficient depth. They have different implications for whether the strategy is salvageable, and the data I retained cannot distinguish them.

9.1 Quote Fading by Faster Participants

The simplest explanation is that other participants reacted to the same Binance move faster than 150 ms and withdrew their quotes before my order arrived. This requires no unusual behaviour: it is ordinary market making. Section 2.3 establishes that the first re-quotes appear within 150–500 ms, which brackets my arrival time, so this mechanism is consistent with the measured timing.

If this is the cause, the problem is latency after all, and the geographic split is the binding constraint. A strategy would need to arrive materially inside 150 ms, which the Asia-to-Europe transit precludes, or predict the move in advance, which Section 8 found infeasible at the required signal strength.

9.2 The Off-Chain to On-Chain Settlement Gap

Polymarket matches orders off-chain and settles them on-chain, which creates a window in which a matched order can be invalidated before it settles. This has since been documented in detail. Shen et al. [1] reconstruct 1,952,440 reverted match-order transactions and identify four attack vectors by which makers can selectively revert filled orders, reporting that during peak hours more than 24.3% of filled orders reverted. A concurrent study [2] analyses 1.8 million reverted transactions over the period from August 2025 to May 2026, which contains my live deployment, and describes the same atomicity gap.

If this is the cause, the displayed size was genuinely resting but not genuinely firm, and no amount of speed would have helped, because the invalidation occurs after matching. It would also mean the failure was a venue property documented independently months later rather than anything specific to my system.

I cannot confirm this from my own data. Establishing it requires cross-referencing my wallet’s transaction history on Polygon against my order log to determine whether unfilled orders were rejected at match time or reverted at settlement. That check is straightforward and remains outstanding; the two outcomes have different signatures and are distinguishable from records that still exist.

9.3 Spoofing

Resting orders may have been posted without intent to trade and cancelled on the approach of marketable flow. Low-participation venues are more susceptible to this, since few actors are needed to shape the visible book.

I have no evidence for this beyond the fill rate itself, which is equally consistent with the two mechanisms above. Spoofing is a claim about intent, and intent cannot be inferred from a fill rate. I raise it because it was my working hypothesis during the deployment, and I note that it was not the best-supported one.

9.4 Why the Paper Harness Could Not Distinguish Them

Under all three mechanisms the pre-trade book looks the same. The mock CLOB simulated fills by walking displayed levels, and displayed levels are exactly what each mechanism leaves intact. Fading and spoofing both produce cancellations that are a reaction to, or concurrent with, the arriving order, so no pre-trade snapshot contains them. The settlement gap produces failures after matching, which a mock that stops at matching cannot see.

Making the mock more realistic in the ways I pursued, consuming displayed liquidity, enforcing real rate limits, and using the real feed, improved API fidelity without touching the fill model. The error was treating the mock’s realism as evidence about fills when its realism was entirely about the interface.

9.5 Adverse Selection Among the Fills That Occurred

One observation constrains the picture further. The fills I did get were close to coin flips despite a backtested 61.4% win rate. If unavailability were independent of the trade’s quality, the 10% that filled should have retained the edge at lower volume. They did not, which suggests selection: liquidity persisted when the resting side was willing to trade, which is disproportionately when that side is better informed or the move is about to reverse.

This also explains why the outcome was flat rather than losing. With fees at 55% of gross, coin-flip fills lose slowly rather than quickly, and at \$5 per order over one week, on a fill rate of one in ten, the resulting loss is small enough to sit inside noise. A longer or larger deployment would likely have shown a small negative rather than a clean zero, and describing the result as “flat” should be read with that in mind.

10 Lessons

10.1 Realisable Liquidity Is a Measurement, Not an Assumption

The strategy's signal was validated thoroughly and its execution surface was assumed. A corrected process treats realisable liquidity as a first-class unknown. Before committing capital: run a campaign of small live probe orders across the target markets and measure empirical fill rate as a function of displayed size, aggressiveness and time of day; compare displayed depth against depth that actually transacts in historical trade prints; and, on a venue with on-chain settlement, check what fraction of matched orders settle. A venue where realised fills fall far short of displayed depth is untradeable by a liquidity taker regardless of signal quality.

The probe campaign would have cost a few days and a few dollars, and it would have produced the 10% figure before the backtest was built rather than after. Most of what came later, the calibration, the modelling, the forecasting fix, was effort spent on a strategy a two-dollar experiment would have shown was untradeable.

10.2 A Fill Model Must Be Able to Decline

A paper harness for a liquidity-taking strategy needs a fill model that can refuse. Even a crude one, filling a displayed level with some probability or applying a haircut calibrated to observed live fill rates, would have collapsed the 61.4% backtest toward break-even and reframed the deployment as a measurement exercise rather than an expected earner. More generally, filling against displayed depth is valid for liquidity-providing strategies and systematically optimistic for liquidity takers.

10.3 A Learned Detector Inherits Whatever the Labels Assume

Moving from hand-tuned gates to a model was the right call on signal quality, but it quietly raised the stakes on the fill assumption. The gates encoded a human belief about what a real move looks like; the model encoded whatever made trades profitable in the training data, and that profitability was measured against the displayed book. So the fill error did not just inflate the backtest, it defined what the detector learned to look for. The later forecasting work hit the same wall for the same reason: every label available to train on was profitability in a market that was not the real one. The general point is that the more of a strategy you learn from data rather than specify by hand, the more it depends on that data's execution model being real, and the less obvious it is when it is not.

10.4 Geography Is Computed First

The 120 ms transatlantic hop was the hard floor of this design. It did not consume the two-second repricing window, but it did place arrival at the same moment as the fastest re-quoters, which is the part of the window that determines whether liquidity is still there. Where the price source and execution venue are far apart, the transit cost should be the first number computed, and compared against the time at which competing participants act rather than against the time the full move completes.

10.5 Calibrate Against the System That Will Run

Two exit timeouts in Section 4 could never fire: the 27 s ETH and XRP timeouts sit behind a 15 s dead-man timer, and SOL's 3 s timeout preempts the 3.5 s minimum hold that convergence needs. Both came from tuning the exit parameters against a replay that did not enforce the live state machine's constraints. Optimising a parameter the deployed system cannot honour produces backtest numbers for a system that does not exist. The dead confidence gate left over from the old scheme is the same mistake in a smaller form: a control that looks active on paper and does nothing in fact.

10.6 Where This Approach Does Work

None of this invalidates cross-venue latency arbitrage as a class. It works when the liquidity being raced toward is firm: deep, professionally market-made venues where displayed size is largely real, settlement is atomic with matching, and cancellation-on-arrival is the exception. The infrastructure here, the two-node split, the minimal UDP protocol, the shared-nothing per-asset pipelines, the mock-to-live harness and the per-asset modelling and calibration stack, transfers to such a venue. What must change is the target.

11 Conclusion

This was a complete, deployed, real-money low-latency trading system built around an empirically validated inefficiency. The signal predicted short-horizon repricing, the backtest was profitable with coherent conditional structure, the infrastructure delivered orders inside the repricing window, and the deployment was cheap, stable and correct. It made no money, because the displayed liquidity it was designed to take was not available to a taker arriving at 150 ms.

What I can establish is that unavailability and its magnitude. What I cannot establish, from the data I kept, is its cause: quote fading by faster participants, the off-chain to on-chain settlement gap since documented at scale by others, or deliberate spoofing are all consistent with a 10% fill rate, and they have different implications for whether the strategy could be repaired. Distinguishing them requires maker-level quote lifecycle data or on-chain settlement records for my own orders, and the second of these is still recoverable.

The engineering artefacts retain their value: a template for two-node systems split across regions, a compact loss-tolerant signalling protocol, a shared-nothing processing model, and a mock-to-live harness with faithful API parity. The more portable lesson concerns validation. Speed is necessary and not sufficient in a latency arbitrage, and whether the liquidity at the end of the race is real is a question to be measured with small probe orders before a system is built around the answer, not diagnosed afterward. Establishing that here cost approximately \$100 of infrastructure, \$5 per trade and one week of live capital, which is the right price for the information.

A System Reference

A.1 Component Map

Table 3: Principal components by node.

Component	Node	Responsibility
BinanceFeed	Tokyo	Binance depth and trade WebSocket ingest; inter-snapshot book estimation
RollingWindow	Tokyo	60 s per-asset metrics (mid, OFI, VWAP, flow, imbalance, price ring)
MovePreFilter	Tokyo	Coarse move-size threshold; discards obvious no-move ticks
SignalModel	Tokyo	Per-asset LightGBM / random-forest detector and score
ExitMonitor	Tokyo	Tracks active signals; emits CLOSE on reversal or timeout
SignalBridge / Sender	Tokyo	Packs 20-byte frames; sends 3 redundant UDP copies; heartbeat
SignalReceiver	Dublin	UDP ingest; sequence-based dedup; dead-man detection
PolyClobFeed	Dublin	Polymarket CLOB book and trade WebSocket; window rotation
MarketWorker (×8)	Dublin	Per-(asset, timeframe) event processing over private ring buffers
trade_logic	Dublin	Ten entry gates, five exit paths
OrderEngine / Executor	Dublin	ECDSA signing (Rust); FAK execution; fill reconciliation
PositionManager / FSM	Dublin	Per-slot state machine; two-phase open/close; risk gating
RateLimiter	Dublin	Dual-window API limiter with close reserve

A.2 Results Summary

Table 4: Replay backtest (displayed-liquidity fill assumption) and live outcome.

Metric	Value
<i>Backtest, \$100 order cap</i>	
Trades	8,576
Recorded sessions	13 (approx. 52 hours, 5-minute markets)
Win rate	61.4%
Median hold	7.6 s
Filled size (derived)	approx. 6 contracts/trade
Gross P&L	+51,350¢ (5.99¢/trade, approx. 1.0¢/contract)
Fees	28,403¢ (3.31¢/trade, 55% of gross)
Net P&L	+22,948¢ (2.68¢/trade, approx. 0.4¢/contract)
Convergence exits	78%
Emergency / reversal / stop	16% / 4% / 2%
Fill rate	approx. 100% (by construction)
<i>Live, \$5 per order</i>	
Duration	approx. 1 week
Fill rate	approx. 10%
Win rate on fills	approx. 50%
Outcome	flat
<i>Measured venue characteristics</i>	
Repricing half-life	approx. 786 ms (median)
First re-quotes	150–500 ms
End-to-end latency	140–150 ms

A.3 Key Parameters

Table 5: Selected system parameters.

Parameter	Value	Where
Assets	BTC, ETH, SOL, XRP	both
Timeframes	5 min (traded), 15 min (untraded)	Dublin
Binance ingest rate	approx. 3,000 updates/s	Tokyo
Detector lookback	25–150 ms	Tokyo
Book feed granularity	100 ms snapshots	Tokyo
Signal packet size	20 bytes ($\times 3$ copies)	wire
Heartbeat interval	2 s	wire
Max signal age gate	500 ms	Dublin
Price-bounds gate	[0.25, 0.80]	Dublin
Order size (paper / live)	\$100 / \$5	Dublin
Exposure caps	\$500 / asset, \$2,000 total	Dublin
Convergence min. hold	3.5 s	Dublin
Emergency dead-man	15 s	Dublin
Instances	$2 \times$ c6a.large (2 vCPU)	AWS
Regions	ap-northeast-1, eu-west-1	AWS
End-to-end latency	140–150 ms	measured
Monthly cost	approx. \$100	AWS

A.4 Data Provenance

Backtest figures derive from two datasets recorded during paper operation: the `regime_trade_log` (8,576 simulated trades) and `edge_signals` (4,892 detected signals with post-signal Poly-market repricing at fixed horizons). The two counts are not in a fixed ratio because they were recorded over overlapping but not identical session sets, and because a signal can produce activity in more than one (asset, timeframe) slot. Figures drawn from each are labelled accordingly and should not be cross-multiplied.

Live figures derive from the Dublin trade log over the deployment week. Fill size was not logged per trade and is recovered from the fee total in Section 7. Maker-level quote lifecycle data was not captured either, and it is the missing piece that keeps Section 9 from settling on a single cause. One check does remain open, though: the wallet’s transaction history on Polygon can be cross-referenced against the order log to see whether unfilled orders were rejected at match time or reverted at settlement, and those two leave different traces. It has not yet been run.

References

- [1] Y. Shen et al., *The Ghosts of Polymarket: When Off-Chain Matches Meet On-Chain Reverts*, arXiv:2606.16852, 2026.
- [2] *Ghost-Filled Orders: Detecting and Testing Atomicity Violations in Non-Custodial Prediction Markets*, arXiv:2609.17902, 2026.